TÍTULO DEL TRABAJO:

DISEÑO Y DESARROLLO DE SERVICIOS WEB - PROYECTO

GA7-220501096-AA5-EV03

NOMBRE DEL AUTOR:

DIEGO ALEXANDER CASTIBLANCO RAMÍREZ

INSTITUCIÓN:

SENA

CURSO:

ANALISIS Y DESARROLLO DE SOFTWARE. (3070454)

PROFESOR:

NELSON GUARIN

FECHA DE ENTREGA:

30 ABRIL 2026

Introducción

El proyecto WiseParking se encuentra en una etapa de modernización arquitectónica fundamental. Este documento detalla la transición técnica de un sistema monolítico tradicional, con vistas renderizadas del lado del servidor mediante Thymeleaf, hacia una arquitectura Full-Stack moderna y completamente desacoplada. El objetivo central de esta fase es la refactorización del backend construido en Spring Boot 3.3.4 y Java 17, transformando sus controladores clásicos en una API RESTful. Esta evolución permitirá estandarizar la comunicación a través de formato JSON y habilitará las configuraciones de seguridad de recursos cruzados (CORS), sentando una base sólida y escalable para la inminente integración de una interfaz de usuario dinámica desarrollada en React JS.

Versión: 1.0.0 **Base URL:** `http://localhost:8081/api` **Descripción:** Servicios web para la gestión de usuarios y control de acceso/salida de vehículos en parqueaderos.

```
19
20  const API_URL = "http://localhost:8081/api/vehiculos";
21
22  useEffect(() => {
23    if (view === 'admin') fetchVehiculos();
24  }, [view]);
25
```

1. Módulo de Autenticación (Auth)

1.1. Registrar Usuario

* **Endpoint**:** `/auth/register`

* **Método HTTP**:** `POST`

* **Descripción**:** Registra un nuevo propietario/administrador de parqueadero en el sistema.

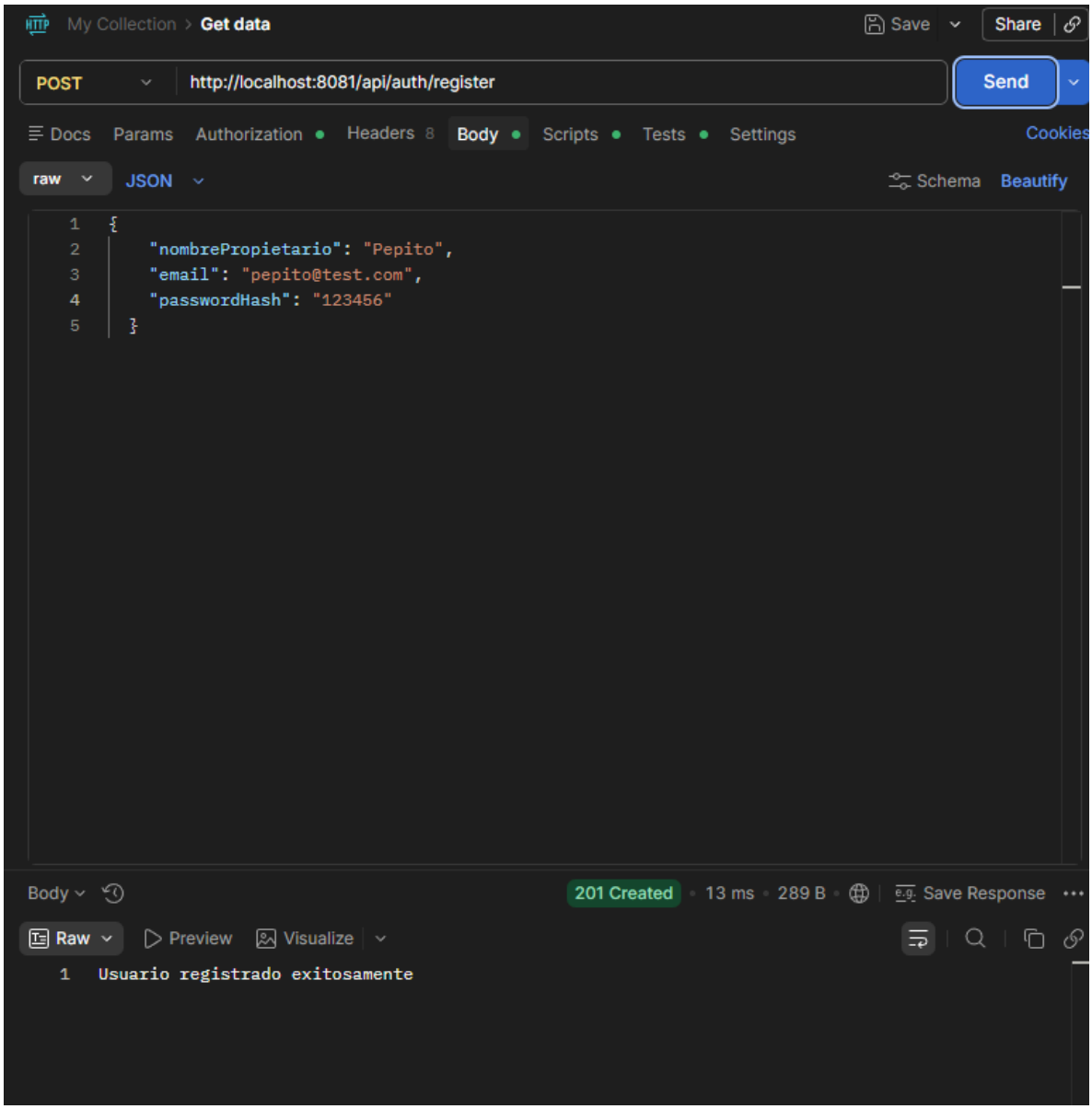
* **Body Request (JSON):**

```
{
  "nombrePropietario": "String",
  "email": "String",
  "passwordHash": "String"
}
```

```
65    e.preventDefault();
66    try {
67      const res = await axios.post("http://localhost:8081/api/auth/
68      alert("✅ " + res.data);
69      setLoginData({ email: '', passwordHash: '' });
70      setView('admin');
71    } catch (err) {
```

Respuestas Esperadas:

- `201 Created`: "Usuario registrado exitosamente"
- `400 Bad Request`: "Error al registrar usuario..."



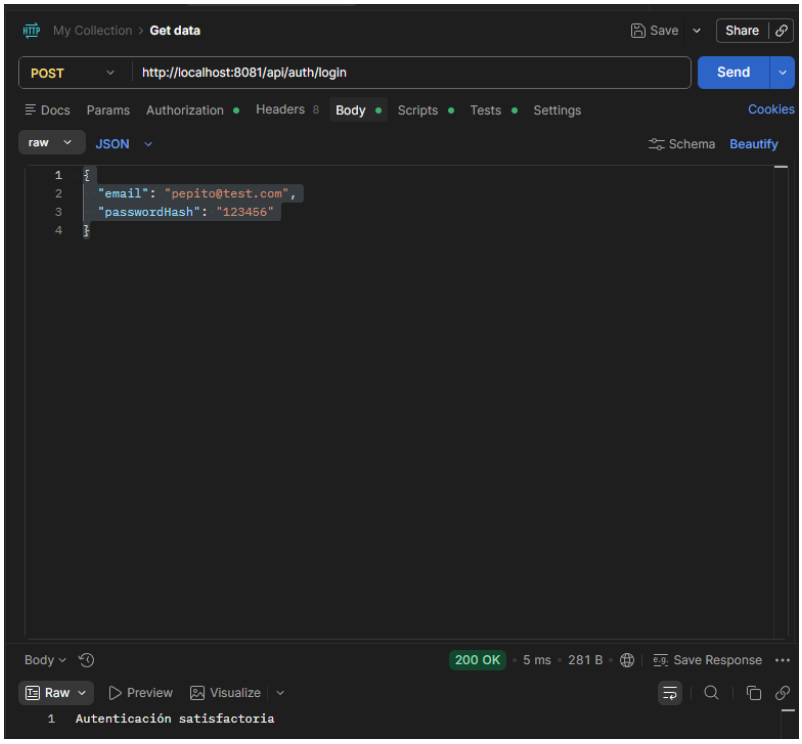
1.2. Iniciar Sesión

- **Endpoint:** `/auth/login`
- **Método HTTP:** `POST`
- **Descripción:** Valida las credenciales de un usuario existente.
- **Body Request (JSON):**

```
{
  "email": "pepito@test.com",
  "passwordHash": "123456"
}
```

Respuestas Esperadas:

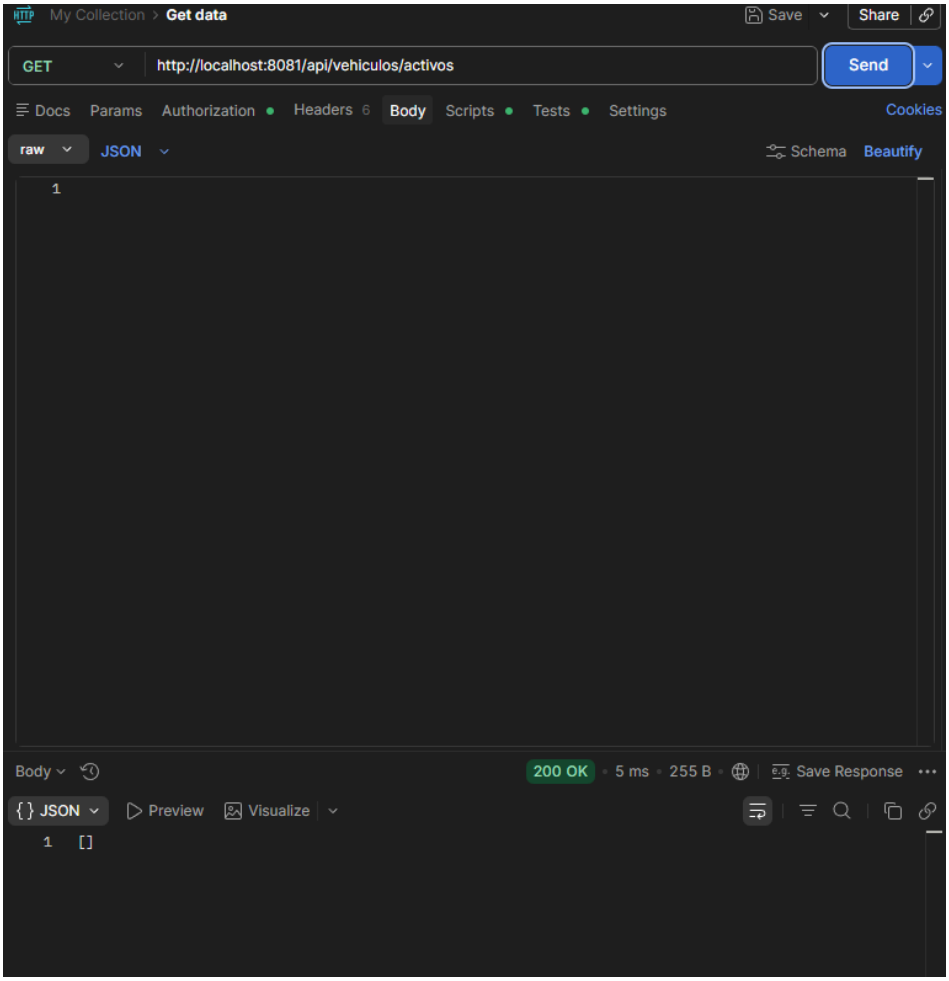
- `200 OK`: "Autenticación satisfactoria"
- `401 Unauthorized`: "Error en la autenticación"



2. Módulo de Gestión de Vehículos

2.1. Listar Vehículos Activos

- **Endpoint:** `/vehiculos/activos`
- **Método HTTP:** `GET`
- **Descripción:** Obtiene una lista de todos los vehículos que se encuentran actualmente dentro del parqueadero (sin hora de salida registrada).
- **Respuestas Esperadas:**
  - `200 OK`: Retorna un Array de objetos JSON con la lista de vehículos.



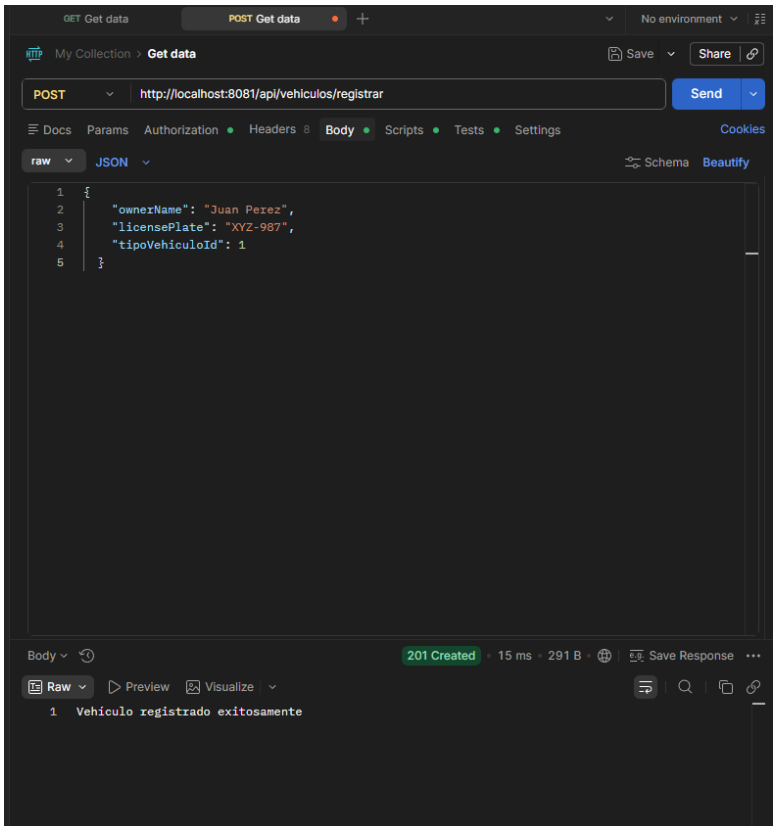
2.2. Registrar Entrada de Vehículo

- **Endpoint:** `/vehiculos/registrar`
- **Método HTTP:** `POST`
- **Descripción:** Registra el ingreso de un vehículo, asignando automáticamente la hora exacta del servidor (`arrivalTime`).
- **Body Request (JSON):**

```
{
  "ownerName": "String",
  "licensePlate": "String",
  "tipoVehiculoId": "Integer (1=Carro, 2=Moto)"
}
```

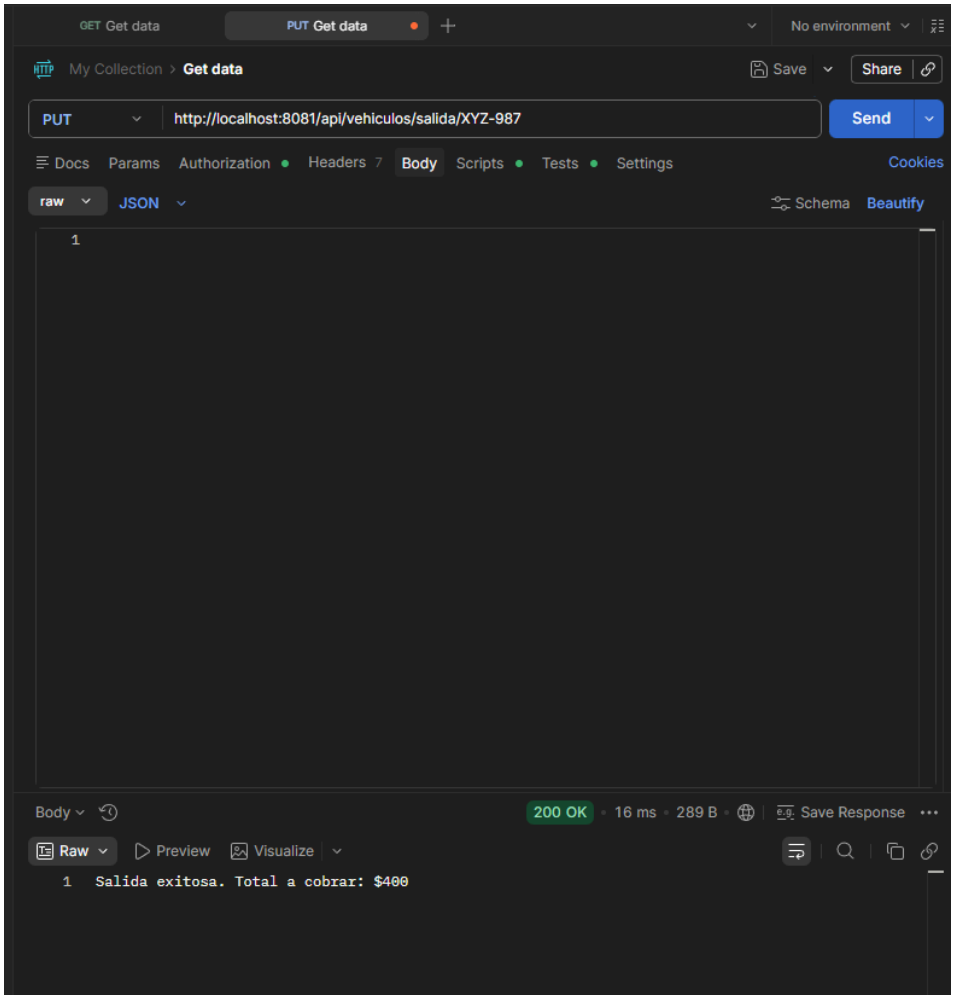
Respuestas Esperadas:

- `201 Created`: "Vehículo registrado exitosamente"
- `500 Internal Server Error`: "Error al registrar el vehículo"



2.3. Procesar Salida y Calcular Tarifa

- **Endpoint:** `/vehiculos/salida/{placa}`
- **Método HTTP:** PUT
- **Descripción:** Registra la hora de salida de un vehículo mediante su placa, calcula el tiempo transcurrido y genera el total a cobrar.
- **Parámetros de Ruta (Path Variables):**
  - `placa` (String): La placa del vehículo a dar salida.
- **Respuestas Esperadas:**
  - `200 OK`: "Salida exitosa. Total a cobrar: $[Monto]"
  - `404 Not Found`: "Error: Vehículo con placa {placa} no encontrado o ya tiene salida."



Conclusion

La refactorización de WiseParking hacia un ecosistema de API REST marca un salto cualitativo en la madurez y escalabilidad del proyecto. Al desvincular la capa de presentación de la lógica de negocio, se ha logrado cumplir con el principio de responsabilidad única, garantizando que el backend de Spring Boot se dedique exclusivamente al procesamiento de datos y la persistencia segura en MySQL. Esta nueva arquitectura desacoplada no solo facilita el mantenimiento del código a largo plazo, sino que consolida un puente de comunicación ágil (vía JSON) indispensable para soportar la interactividad y el alto rendimiento que exige el nuevo frontend en React. El sistema ahora está preparado para operar bajo los estándares de la industria moderna.